Jenkins CI/CD — Hands-on Assignment

AI-Driven Software Engineering & Project Management (24CS3502) · Unit 3 · Department of CSE, Dayananda Sagar University

Assignment details

This is a self-learning assignment. Jenkins has not been covered in class, so this sheet explains each idea before you use it. You will install Jenkins inside your Ubuntu virtual machine, then build up step by step: simple jobs first, then a full CI/CD pipeline that pulls code from GitHub, tests it, packages it, builds a Docker image and deploys it. Every required step is proved with a screenshot in a report submitted as a single PDF file.

Item	Detail
Course	AI-Driven Software Engineering & Project Management (24CS3502)
Unit	Unit 3 — Continuous Integration / Continuous Delivery with Jenkins
Mode	Individual
Where you work	The Ubuntu 24.04 VM in VMware (`usn-vm`) from the Docker assignment. If you have not made it yet, follow Experiment A1 of that assignment first
VM settings	At least 4 GB RAM, 2 CPUs, 10 GB free disk inside the VM
Accounts needed	A free GitHub account (github.com)
Experiments	J1–J13
Maximum marks	20
Submission	One PDF file, uploaded to a Google Drive link shared by the instructor
Due date	The Drive link stays open for **2 days** after it is shared; the deadline is when the link closes

Learning outcomes

After completing this assignment, the student will be able to:

1. Explain Continuous Integration (CI) and Continuous Delivery (CD) and the role of Jenkins in them.
2. Install, unlock and configure Jenkins, and manage its plugins, users and credentials.
3. Create freestyle jobs with shell build steps, parameters and build triggers.
4. Connect a Jenkins job to a GitHub repository and build automatically when code changes.
5. Write a declarative pipeline (`Jenkinsfile`) with build, test, package and deploy stages.
6. Publish test reports and archive build artifacts, and diagnose a failed build.
7. Build and run a Docker image from a Jenkins pipeline.

Screenshot rules (read carefully)

- In Experiment J2 you create a Jenkins admin user whose **username is your USN**. Every Jenkins screenshot must show that username in the top-right corner of the Jenkins page.
- Browser screenshots must show the address bar (`localhost:8080/...`) and the VMware window frame with the VM name `usn-vm`.
- Every terminal screenshot must show your own username in the prompt.
- Use a terminal and browser zoom large enough that the text is readable in the PDF without zooming in.
- Crop to the relevant window. Paste screenshots as images into your report (no more than 4 per page). Do not take photos of the screen with a phone.
- Below each screenshot, write a 1–2 line caption in your own words explaining what it shows.
- In every name below, replace `usn` with your own USN in lowercase (e.g. `1ds24cs001`).

Background: what you need to know first

Why CI/CD?

Imagine 10 students working on one project. Each writes code for a week, then they try to combine everything the night before the deadline. Nothing works, and nobody knows whose change broke it. This is called “integration hell”.

Continuous Integration (CI) fixes this: every time anyone pushes code, a server automatically **builds** it and **runs the**

tests within minutes. If something breaks, the team knows immediately and knows exactly which change caused it.

Continuous Delivery (CD) goes one step further: every change that passes the tests is automatically **packaged** and made ready to **deploy**, often with one approval click.

Analogy. CI/CD is like an automated quality-check line in a factory. Each new part (code change) goes down the line: it is assembled (build), inspected (test), boxed (package) and shipped (deploy). A faulty part is stopped at the inspection station and never reaches the customer.

What is Jenkins?

Jenkins is a free, open-source **automation server**, written in Java, that runs CI/CD pipelines. You open it in a web browser (default address `http://localhost:8080`). Almost every feature in Jenkins comes from a **plugin**; there are over 1,800 of them, for Git, Docker, testing tools, notifications and more.

The CI/CD flow you will build

1. **Developer** commits code to a **GitHub** repository.
2. **Jenkins** notices the change (by polling GitHub every 2 minutes).
3. **Build** — Jenkins checks out the code and compiles it.
4. **Test** — Jenkins runs the automated tests and publishes a test report.
5. **Package** — Jenkins creates a deliverable file (an *artifact*) and stores it.
6. **Docker image** — Jenkins builds a container image of the application.
7. **Deploy** — after a human clicks *Approve*, Jenkins copies the application to a “staging” folder.

If any step fails, the pipeline stops there and the build turns **red**.

Key terms

Term	Meaning
Controller	The main Jenkins server: it shows the web interface, stores configuration and schedules builds (older documents call it “master”)
Agent / Node	A machine that actually runs builds. In this assignment the controller also runs the builds (the **Built-In Node**)
Executor	One build “slot” on a node. 2 executors = 2 builds can run at the same time
Job / Project / Item	A configured task, for example “pull code and run tests”
Build	One run of a job, numbered #1, #2, #3 …
Console Output	The full log of a build — the first place to look when something fails
Workspace	The folder where a job’s files are checked out and built (`/var/lib/jenkins/workspace/<job-name>`)
Freestyle project	A job configured by filling in web forms. Easy, but hard to version-control
Pipeline	A job defined as code, in a language based on Groovy, made of **stages** and **steps**
Jenkinsfile	A text file holding the pipeline code, stored in the Git repository with the application
Trigger	What starts a build: a click, a timer, a code change, or another job finishing
Artifact	A file produced by a build (a `.jar`, `.tar.gz`, report …) that Jenkins keeps
Credentials	Passwords, tokens and keys stored securely in Jenkins and hidden in logs
Plugin	An add-on that gives Jenkins a new feature

Build status colours

Status	Icon	Meaning
Success	Green tick	All steps completed
Unstable	Yellow / orange	Build completed but some tests failed (or a quality check failed)
Failure	Red cross	A step failed; the build stopped
Aborted	Grey	Someone cancelled it, or it timed out

Part 1 — Install and explore Jenkins

Experiment J1 — Install Java and Jenkins in the Ubuntu VM

Aim: Install Jenkins as a service on Ubuntu.

Jenkins needs **Java 21**. Open a terminal in `usn-vm` and run:

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre git python3-pytest
java -version
```

(`git` and `python3-pytest` are needed in later experiments.)

Add the official Jenkins LTS (Long-Term Support) repository and install Jenkins:

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" \
  | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install -y jenkins
```

If `apt update` shows a key or signature error, the key file name has changed: copy the current commands from **jenkins.io/doc/book/installing/linux** (Debian/Ubuntu section, LTS release).

Check that the service is running:

```
sudo systemctl status jenkins
sudo systemctl enable jenkins
jenkins --version
```

- **SS J1.1** Output of `java -version` showing version 21.
- **SS J1.2** Output of `systemctl status jenkins` showing *active (running)* and `jenkins --version`.
- **Write:** Jenkins runs as a Linux user called `jenkins`, and its home folder is `/var/lib/jenkins`. Run `id jenkins` and `sudo ls /var/lib/jenkins` and describe what you see.

Experiment J2 — Unlock Jenkins and complete the setup wizard

Aim: Finish the first-time setup in the browser.

1. Open **Firefox inside the VM** and go to **http://localhost:8080**. The *Unlock Jenkins* page appears.
2. Print the one-time admin password and paste it into the page:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

3. On *Customize Jenkins*, click **Install suggested plugins** and wait for all plugins to install (5–15 minutes).
4. On *Create First Admin User*, fill in:
    - Username: **your USN** (e.g. `1ds24cs001`)
    - Password: a password you will remember
    - Full name: your full name
    - E-mail: your college e-mail
5. On *Instance Configuration*, keep `http://localhost:8080/` and click **Save and Finish → Start using Jenkins**.

- **SS J2.1** The *Getting Started* page while suggested plugins install.
- **SS J2.2** The *Create First Admin User* form filled with your details (hide the password).
- **SS J2.3** The Jenkins Dashboard (“Welcome to Jenkins!”) with your USN in the top-right corner.

Experiment J3 — Tour of the Jenkins interface

Aim: Learn where things are before creating jobs.

Visit each of these and note what it does:

Where	What to look at
Dashboard (left menu)	**New Item**, **Build History**, **Build Queue**, **Build Executor Status**
Manage Jenkins → **System**	Jenkins URL, number of executors, global settings
Manage Jenkins → **Tools**	Where Git, JDK, Maven etc. can be configured
Manage Jenkins → **Plugins**	Installed, Available and Updates tabs
Manage Jenkins → **Nodes**	The Built-In Node and its executors
Manage Jenkins → **Credentials**	Stored secrets (empty for now)
Manage Jenkins → **System Information**	Java version, OS, environment
http://localhost:8080/env-vars.html	List of environment variables every build gets (`BUILD_NUMBER`, `JOB_NAME`, `WORKSPACE` …)

- **SS J3.1** The *Manage Jenkins* page.
- **SS J3.2** Manage Jenkins → Nodes showing the Built-In Node with its number of executors.
- **SS J3.3** Manage Jenkins → Plugins → Installed plugins (showing at least *Git* and *Pipeline*).
- **Write:** In one line each, explain *controller*, *node*, *executor* and *plugin* using what you saw.

Part 2 — Freestyle jobs

Experiment J4 — Your first freestyle job

Aim: Create a job that runs shell commands and read its console output.

1. Dashboard → **New Item** → name `usn-hello-freestyle` → select **Freestyle project** → **OK**.
2. Description: `My first Jenkins job`.
3. Scroll to **Build Steps** → **Add build step** → **Execute shell**. Enter (replace name and USN):

```
echo "Hello from Jenkins - Name: Your Name, USN: 1DS24CS001"
echo "Job name    : $JOB_NAME"
echo "Build number: $BUILD_NUMBER"
echo "Workspace   : $WORKSPACE"
echo "Running as  : $(whoami) on $(hostname)"
date > build-info.txt
ls -l
```

4. **Save**, then click **Build Now** three times.
5. Click build **#3** → **Console Output**.
6. In the VM terminal, look at the workspace:

```
sudo ls -l /var/lib/jenkins/workspace/usn-hello-freestyle
```

- **SS J4.1** The *Execute shell* build step in the job configuration.
- **SS J4.2** Console Output of build #3 ending with *Finished: SUCCESS*.
- **SS J4.3** The job page showing three builds in *Builds* history.
- **Write:** Which user ran the commands? Why is `build-info.txt` still in the workspace after the build finished?

Experiment J5 — Parameterized job

Aim: Let the person starting the build give input values.

1. New Item → `usn-param-job` → **Freestyle project**.
2. Tick **This project is parameterized** and add:
   - **String Parameter** — Name: `STUDENT_NAME`, Default: your name
   - **Choice Parameter** — Name: `ENVIRONMENT`, Choices (one per line): `dev`, `test`, `prod`
   - **Boolean Parameter** — Name: `RUN_TESTS`, Default: ticked
3. Build Steps → Execute shell:

```
echo "Build requested by: $STUDENT_NAME"
echo "Target environment: $ENVIRONMENT"
if [ "$RUN_TESTS" = "true" ]; then
  echo "Running tests..."
else
  echo "Tests skipped"
fi
if [ "$ENVIRONMENT" = "prod" ]; then
  echo "WARNING: deploying to production"
fi
```

4. Save → **Build with Parameters**. Run it twice: once with `dev` + tests ticked, once with `prod` + tests unticked.

> - **SS J5.1** The *Build with Parameters* page.
> - **SS J5.2** Console Output of both builds.
> - **Write:** Give one real situation where a parameterized build is useful.

Experiment J6 — Build triggers: timer and job chaining

Aim: Start builds automatically instead of clicking *Build Now*.

(a) Build periodically. Open `usn-hello-freestyle` → **Configure** → *Triggers* → tick **Build periodically** → Schedule: `H/5 * * * *` → Save. Wait about 10 minutes.

Jenkins schedules use **cron syntax** — five fields:

Field	MINUTE	HOUR	DAY OF MONTH	MONTH	DAY OF WEEK
Range	0–59	0–23	1–31	1–12	0–7 (0 and 7 = Sunday)

`H/5 * * * *` = every 5 minutes. `H 2 * * 1-5` = once between 2:00 and 2:59 a.m., Monday to Friday. `H` lets Jenkins pick a fixed time so that not all jobs start at the same moment.

(b) Job chaining (upstream → downstream).

1. New Item → `usn-downstream` → Freestyle project.
2. Triggers → tick **Build after other projects are built** → Projects to watch: `usn-hello-freestyle` → *Trigger only if build is stable*.
3. Execute shell: `echo "Started because usn-hello-freestyle succeeded"`
4. Save. Run `usn-hello-freestyle` once with **Build Now**.

After taking the screenshots, untick *Build periodically* so the job stops running every 5 minutes.

> - **SS J6.1** Build History showing builds whose cause says *Started by timer*.
> - **SS J6.2** Console Output of `usn-downstream` showing *Started by upstream project "usn-hello-freestyle"*.
> - **Write:** Write the cron expression for "every day at 11 p.m." and for "every Monday at 9 a.m.". Explain what upstream and downstream mean.

Part 3 — Jenkins with GitHub and pipelines

Experiment J7 — Create a GitHub repository and build from it

Aim: Make Jenkins fetch code from GitHub and test it on every change.

Step 1 — Create the repository. Sign in to github.com → **New repository** → name `usn-jenkins-lab` → **Public** → tick *Add a README file* → **Create repository**.

Use **Add file → Create new file** on GitHub to add these two files (commit each one to `main`).

`calculator.py`:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

`test_calculator.py`:

```
import pytest
from calculator import add, subtract, multiply, divide

def test_add():
    assert add(2, 3) == 5

def test_subtract():
    assert subtract(10, 4) == 6

def test_multiply():
    assert multiply(3, 4) == 12

def test_divide():
    assert divide(10, 2) == 5

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(5, 0)
```

Step 2 — Freestyle job connected to Git.

1. New Item → `usn-git-freestyle` → Freestyle project.
2. **Source Code Management** → **Git** → Repository URL: `https://github.com/<your-github-username>/usn-jenkins-lab.git` → Branch Specifier: `*/main`. (No credentials needed for a public repository.)
3. **Triggers** → tick **Poll SCM** → Schedule: `H/2 * * * *` (check GitHub every 2 minutes).
4. Build Steps → Execute shell:

```
ls -l
python3 -m pytest -v
```

5. Save → **Build Now**.
6. On GitHub, edit `README.md` (add a line with your name) and commit. Within about 2 minutes Jenkins starts a new build on its own.

> - **SS J7.1** Your GitHub repository page showing the three files.
> - **SS J7.2** Source Code Management section of the job configuration.
> - **SS J7.3** Console Output showing the Git checkout and `5 passed`.
> - **SS J7.4** The automatic build showing *Started by an SCM change*, and the job's **Git Polling Log**.
> - **Write:** Your VM is behind NAT, so we used *Poll SCM*. In companies, GitHub usually *pushes* a notification to Jenkins using a **webhook**. Explain the difference between polling and a webhook.

Experiment J8 — Your first pipeline (script written in Jenkins)

Aim: Learn declarative pipeline syntax: `pipeline`, `agent`, `stages`, `stage`, `steps`, `post`.

1. New Item → `usn-first-pipeline` → select **Pipeline** → OK.
2. Scroll to **Pipeline** → Definition: **Pipeline script** → paste:

```
pipeline {
    agent any                       // run on any available node
    environment {
        STUDENT = 'Your Name (1DS24CS001)'
    }
    stages {
        stage('Hello') {
            steps {
                echo "Hello, ${env.STUDENT}"
            }
        }
        stage('System info') {
            steps {
                sh 'uname -a'
                sh 'java -version'
                sh 'python3 --version'
            }
        }
        stage('Parallel checks') {
            parallel {
                stage('Check A') {
                    steps { sh 'sleep 5; echo "Check A done"' }
                }
                stage('Check B') {
                    steps { sh 'sleep 5; echo "Check B done"' }
                }
            }
        }
    }
    post {
        always  { echo 'Pipeline finished' }
        success { echo 'All stages passed' }
    }
}
```

3. Save → **Build Now**. Open the build and view the **stage graph** (shown as *Pipeline Overview* / stage view on the build page; if you see neither, install the *Pipeline Graph View* plugin in J13 and return).
4. Click **Pipeline Syntax** (link under the script box). This is the **Snippet Generator**: choose a step such as `sh` or `echo`, fill in the form and click *Generate Pipeline Script* to see the code.

> - **SS J8.1** The stage graph showing Hello, System info and the two parallel checks, all green.
> - **SS J8.2** Console Output showing your name and the `post` messages.
> - **SS J8.3** The Snippet Generator with a generated `sh` step.
> - **Write:** Explain each keyword: `pipeline`, `agent`, `environment`, `stages`, `stage`, `steps`, `parallel`, `post`. How long did *Parallel checks* take, and why not 10 seconds?

Experiment J9 — Full CI/CD pipeline from a Jenkinsfile

Aim: Keep the pipeline in GitHub (“pipeline as code”) and run build → test → package → deploy.

Step 1. In your GitHub repository, create a file named exactly `Jenkinsfile` (no extension):

```
pipeline {
    agent any
    options {
        timestamps()
        timeout(time: 15, unit: 'MINUTES')
    }
    triggers {
        pollSCM('H/2 * * * *')
    }
    environment {
        APP_NAME = 'usn-calculator'
    }
    stages {
        stage('Build') {
            steps {
                sh 'python3 -m py_compile calculator.py'
                echo 'Syntax check passed'
            }
        }
        stage('Test') {
            steps {
                sh 'python3 -m pytest -v --junitxml=test-results.xml'
            }
        }
        stage('Package') {
            steps {
                sh 'mkdir -p dist'
                sh 'tar -czf dist/${APP_NAME}-${BUILD_NUMBER}.tar.gz calculator.py'
                archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
            }
        }
        stage('Deploy to staging') {
            steps {
                input message: 'Deploy to staging?', ok: 'Deploy'
                sh 'mkdir -p /tmp/usn-staging'
                sh 'cp calculator.py /tmp/usn-staging/'
                sh 'ls -l /tmp/usn-staging'
            }
        }
    }
    post {
        always {
            junit allowEmptyResults: true, testResults: 'test-results.xml'
        }
        success {
            echo "Build ${env.BUILD_NUMBER} of ${env.APP_NAME} passed"
        }
        failure {
            echo "Build ${env.BUILD_NUMBER} failed - read the Console Output"
        }
    }
}
```

Step 2. In Jenkins: New Item → `usn-ci-pipeline` → **Pipeline** → in the *Pipeline* section:

- Definition: **Pipeline script from SCM**
- SCM: **Git** → Repository URL: your repository URL → Branch Specifier: `*/main`
- Script Path: `Jenkinsfile`

Step 3. Save → **Build Now**. When the pipeline reaches *Deploy to staging*, it pauses and asks for approval; click **Deploy** (in the stage graph or the Console Output).

Step 4. Open the finished build and look at **Test Result** (or *Tests*) and **Build Artifacts**. Run **Build Now** once more so the job page shows a test-trend graph.

> - **SS J9.1** Your `Jenkinsfile` in the GitHub repository.
> - **SS J9.2** The pipeline paused at *Deploy to staging?* waiting for approval.
> - **SS J9.3** The stage graph with all four stages green.
> - **SS J9.4** The *Test Result* page showing 5 tests passed.
> - **SS J9.5** The build page showing the archived artifact `usn-calculator-<n>.tar.gz`.
> - **Write:** Why is it better to keep the pipeline in a `Jenkinsfile` in Git than to type it into Jenkins (J8)? What is the purpose of the `input` step, and how does it relate to Continuous *Delivery*?

Experiment J10 — Break the build, then fix it

Aim: See how CI catches a bad change and how to find the cause.

1. On GitHub, edit `test_calculator.py` and change `assert add(2, 3) == 5` to `assert add(2, 3) == 6`. Commit with the message `Break test on purpose - usn`.
2. Wait about 2 minutes. `usn-ci-pipeline` starts automatically (it polls GitHub) and the Test stage fails.
3. Open the failed build → **Console Output** and **Test Result** to find which test failed and why.
4. On GitHub, change it back to `== 5`. Commit with the message `Fix test - usn`. The next build should be green again.

> - **SS J10.1** The stage graph of the failed build (Test red; Package and Deploy not run).
> - **SS J10.2** Console Output or Test Result showing `test_add` failed with `assert 5 == 6`.
> - **SS J10.3** The job page *Builds* list showing the red build followed by a green build, with the commit messages under *Changes*.
> - **Write:** Which stages ran in the failed build, and why did the pipeline stop where it did? How does Jenkins show whose commit broke the build, and why is finding this out within 2 minutes valuable?

Part 4 — Credentials, Docker and administration

Experiment J11 — Store and use a secret with Credentials

Aim: Use a password or token in a build without exposing it.

1. Manage Jenkins → **Credentials** → *System* → **Global credentials (unrestricted)** → **Add Credentials**.
   - Kind: **Secret text**
   - Secret: any made-up value, e.g. `MyFakeToken123`
   - ID: `usn-secret`
   - Description: `Demo secret for assignment`
2. New Item → `usn-credentials-demo` → Pipeline → Pipeline script:

```
pipeline {
    agent any
    stages {
        stage('Use secret') {
            steps {
                withCredentials([string(credentialsId: 'usn-secret', variable: 'MY_SECRET')]) {
                    sh 'echo "The secret is: $MY_SECRET"'
                    sh 'echo "The secret has ${#MY_SECRET} characters"'
                }
            }
        }
    }
}
```

3. Build Now → Console Output.

> - **SS J11.1** The Global credentials page listing `usn-secret` (the secret value is not shown).
> - **SS J11.2** Console Output where the secret is printed as `****` while its length is shown.
> - **Write:** Why should passwords and API tokens never be written directly in a Jenkinsfile in GitHub?

Experiment J12 — Build and run a Docker image from the pipeline

Aim: Combine Jenkins with Docker (from your Docker assignment).

Docker Engine must already be installed in `usn-vm` (Docker assignment, Experiment A2). Allow the `jenkins` user to run Docker, then restart Jenkins:

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Add a file named `Dockerfile` to your GitHub repository:

```
FROM python:3.12-slim
WORKDIR /app
COPY calculator.py test_calculator.py ./
RUN pip install --no-cache-dir pytest
CMD ["python", "-m", "pytest", "-v"]
```

Edit your `Jenkinsfile` on GitHub and add this stage **after** the `Package` stage and **before** `Deploy to staging`:

```
        stage('Docker image') {
            steps {
                sh 'docker build -t ${APP_NAME}:${BUILD_NUMBER} .'
                sh 'docker run --rm ${APP_NAME}:${BUILD_NUMBER}'
                sh 'docker images ${APP_NAME}'
            }
        }
```

Commit. The pipeline runs automatically; approve the deploy step when asked. Then in the VM terminal run `docker images usn-calculator`.

- **SS J12.1** The stage graph showing the new *Docker image* stage in green.
- **SS J12.2** Console Output of the Docker stage showing the build steps and the tests passing inside the container.
- **SS J12.3** VM terminal output of `docker images usn-calculator` listing one tag per build number.
- **Write:** Why do teams build a Docker image in their CI pipeline instead of copying source files to the server?

Experiment J13 — Users, plugins and maintenance

Aim: Do basic Jenkins administration.

1. **Create a second user.** Manage Jenkins → **Users** → **Create User** → username `usn-dev`. Open a Firefox **private window**, go to `localhost:8080` and log in as `usn-dev`.
2. **Security settings.** Manage Jenkins → **Security** → look at *Security Realm* and *Authorization*. Note which option is selected (the default is “Logged-in users can do anything”).
3. **Install a plugin.** Manage Jenkins → **Plugins** → **Available plugins** → search `Pipeline Graph View` → install it if it is not already installed (if installed, install `Dark Theme` instead) → tick *Restart Jenkins when installation is complete*.
4. **Backup knowledge.** In the VM terminal:

```
sudo du -sh /var/lib/jenkins
sudo ls /var/lib/jenkins/jobs
```

- **SS J13.1** The Users list showing your USN account and `usn-dev`.
- **SS J13.2** Jenkins in the private window, logged in as `usn-dev` (top-right).
- **SS J13.3** Plugins → *Download progress* page showing the plugin installed successfully.
- **SS J13.4** Terminal output of the two commands above.
- **Write:** All jobs, builds and settings live in `/var/lib/jenkins`. What would you back up to restore this Jenkins on a new machine?

Troubleshooting

Problem	Fix
`localhost:8080` shows nginx or “Address already in use”	A container from the Docker assignment is using port 8080: `docker rm -f usn-web`, then `sudo systemctl restart jenkins`
Jenkins page does not load	`sudo systemctl status jenkins`; read errors with `sudo journalctl -u jenkins -n 50`
`apt update` key/signature error for Jenkins	The repository key has changed; copy the current commands from jenkins.io/doc/book/installing/linux
Suggested plugins fail to install	Check internet in the VM (`ping -c 3 google.com`); click *Retry*
Forgot admin password	Ask the instructor, or reset Jenkins completely: `sudo apt purge jenkins && sudo rm -rf /var/lib/jenkins`, then repeat J1–J2
`python3: No module named pytest`	`sudo apt install -y python3-pytest`
`Couldn't find any revision to build`	Branch Specifier must match GitHub’s branch: `*/main` (not `*/master`)
Poll SCM never triggers	Check the job’s *Git Polling Log*; make sure the commit was on `main`
`permission denied ... docker.sock` in pipeline	`sudo usermod -aG docker jenkins` then `sudo systemctl restart jenkins`
Pipeline stuck at *Deploy to staging?*	It is waiting for you: click **Deploy** in the stage graph or Console Output
VM becomes very slow	Give the VM 4 GB RAM; stop unused containers (`docker ps`, `docker stop <name>`)

Report format and evaluation

The PDF report is graded out of 20, mostly on whether every required screenshot is present, readable and explained in your own words.

What the report must contain (in this order)

1. **Cover page** — college name, department, course name and code, assignment title ("Jenkins CI/CD — Hands-on Assignment"), your name, USN, section, date of submission.
2. **System details** — one table: VM settings (RAM/CPU), Ubuntu version, Java version, Jenkins version, Docker version, your GitHub repository URL.
3. **One section per experiment (J1–J13)** — heading with experiment number and aim, the screenshots labelled exactly as in this sheet (SS J1.1, SS J1.2 …), a 1–2 line caption under each, and the answers to the **Write** questions.
4. **Final Jenkinsfile** — the complete `Jenkinsfile` from your repository (with the Docker stage), copied as text.
5. **Pipeline diagram** — draw (by hand or with any tool) the stages of your final pipeline from GitHub commit to deploy.
6. **Conclusion** — half a page: what you learned and one problem you faced and how you fixed it.

Expected length: 18–25 pages.

How to submit

1. Prepare the report in Word, Google Docs or LibreOffice, with screenshots pasted in, then export it as **one PDF file** (File → Save as / Download → PDF).
2. Name the file `USN_Name_Jenkins.pdf`, for example `1DS24CS001_RahulKumar_Jenkins.pdf`.
3. Keep the file under **20 MB**. If it is larger, compress the images before exporting.
4. Open the PDF once before uploading and check that every page and screenshot is readable.
5. The instructor will share a **Google Drive link** later. Upload your PDF there. The link stays open for **2 days** after it is shared; submissions are not accepted after it closes.
6. Upload only the PDF. Word files, image files, ZIP files and links to your own Drive are not accepted.
7. Keep your GitHub repository `usn-jenkins-lab` **public** until marks are announced; it may be checked.

Evaluation rubric

Criterion	Marks
J1–J3: Jenkins installed, setup completed, interface explored	3
J4–J6: freestyle job, parameters, timer and chained triggers	4
J7–J10: GitHub integration, pipelines, Jenkinsfile CI/CD, failed and fixed build	6
J11–J13: credentials, Docker stage, users and plugins	3
Answers to Write questions and pipeline diagram (own words, correct)	2
Report quality: readable screenshots, labels, captions, cover page	2
Total	**20**

Academic integrity

- Screenshots without your USN as the Jenkins username, in job names, or in the repository name will get **zero** for that experiment.
- Identical screenshots or answers across two reports will be treated as copying; both reports get zero.
- If your laptop cannot run the VM (e.g. less than 8 GB RAM, or virtualization blocked), inform the instructor as soon as possible so you can use a lab system.

Learn more (optional)

- Jenkins User Handbook: jenkins.io/doc/book
- Pipeline syntax reference: jenkins.io/doc/book/pipeline/syntax
- Jenkins tutorial — build a Python app: jenkins.io/doc/tutorials